MAHESHA S



📍 Mysore, India 570022
📞 +91 98802 77334
✉️ mahesha.shivamallappa@gmail.com

🛠️: Compilers • Computer Architecture • AI Accelerator • GPU • LLVM • MLIR

This detailed resume is organized so you can explore my 20+ year compiler engineering career, along with my perspective on leadership, at whatever depth is useful to you — from a brief introduction about myself to a fully detailed technical view of my work. Click a heading below to jump straight to the section you want to explore.

1	About Me	A brief introduction to who I am, my career journey, highlights, and education.
2	Career Timeline	A transparent record of employment dates, positions, transitions, and career breaks — for anyone verifying continuity.
3	Engineering Portfolio	A detailed, problem-by-problem account of the technical work behind each role — for anyone looking for technical depth.
4	LinkedIn Profile	A view into professional activity and network — for anyone who wants the broader picture.
5	Technical Papers	Published technical papers and write-ups — for anyone interested in the technical writing behind the work.
6	My Thoughts About Leadership	A personal essay on what genuine leadership demands — for anyone curious about the philosophy behind how I lead and mentor.

About Me

A brief introduction to who I am and the journey so far.

WHO I AM

I'm a compiler engineer with over 20 years of experience building production compilers across the stack—spanning Clang, MLIR, and LLVM—for CPU, GPU, and AI accelerator architectures. What I enjoy most is taking a hard, often ambiguous problem, understanding it from first principles, and building a solution from scratch, and I have never hesitated to learn a new technology to whatever depth the work demands. Along the way, I have led in practice without a formal management title—as technical lead guiding a team new to compilers at Siemens, ramping up my team on GPU architecture at Intel, and mentoring the engineers around me—and I find just as much satisfaction in helping engineers grow into strong technical leaders.

MY JOURNEY

After graduating in 2003, I wanted to get into research, so I taught discrete mathematics and C programming at an engineering college in Mysore for about a year. Family circumstances then meant I needed to step into industry, so I set research aside and chose compilers—a rare intersection of rigorous theory and hands-on engineering on real hardware.

Since then, I have worked on compilers for DSPs, CPUs, PLCs, GPUs, and AI accelerators at DACS Software, HP, Siemens, AMD, Samsung, and Intel, with a brief detour into distributed systems and ML (Jivox, Qubole) before returning to compilers in 2019.

My career has had its share of ups and downs—roles that ended in corporate restructuring, and a career break of over a year for family responsibilities and long-term career reflection. The Career Timeline records all of it plainly. What has carried me through is curiosity, a positive frame of mind, and the courage to keep moving forward.

How I think about leading and mentoring people, shaped along this journey, is captured in my essay on leadership.

For the full story of my journey, read my Brief Professional Bio.

SELECTED HIGHLIGHTS

- Technical papers — Distilled 20+ years of hands-on compiler work into four in-depth technical papers: how GPUs evolved and how they work, branch divergence in SIMT machines, systolic arrays in modern AI matrix engines, and modern low-precision numerical formats for AI hardware.
- Intel — As sole owner, designed, implemented, and shipped compiler support for the AI accelerator's widened 44-bit tensor dimensions, exposing each as a standard 64-bit type so the existing programming model stayed intact.
- Intel — As sole owner, designed and implemented end-to-end FP8/FP4 low-precision support—scale computation, quantization, and scaled-MMA lowering—in the MLIR Fuser, an MLIR-based GPU fusion compiler.
- AMD — Stabilized HIP-Clang, the Clang-based HIP C++ compiler for AMD GPUs, and partnered with AMD's PyTorch team to make it the default compiler for PyTorch's AMD GPU backend; as sole owner, built Cooperative Groups device-library support.
- AMD — As sole owner, designed and implemented an AMDGPU shared-memory (LDS) optimization pass merged into upstream LLVM (later disabled by default after OpenMP regressions).
- Samsung — As sole owner, designed and implemented a data-flow-analysis-based pointer-range analysis algorithm in the OpenCL compiler's random test generator (randomizer), enabling it to produce safe, semantically valid pointer expressions; later revisited and published it as a technical paper.
- Siemens — As technical lead, owned the full lifecycle and guided a team with no prior compiler background to deliver a compiler front-end for the Structured Text language used to program Siemens PLCs.
- Siemens — In exploratory research with Siemens teams across multiple global locations, investigated whether workloads could be automatically analyzed and retargeted, via source-to-source compilation, onto heterogeneous hardware such as GPUs; also evaluated Intel's MIC co-processor with Intel's research team, documented in a technical paper.
- HP — Worked with a small group of senior engineers to design and successfully implement the ABI-compatibility bridge that lowered EDG's target-neutral ASTs into HP's native PA-RISC ASTs, moving HP's C++ compiler to the industry-standard EDG front-end; documented in a technical paper.

EDUCATION

MASTER'S University of Mysore Jul 2000 – Jun 2003	Master's in Computer Science
BACHELOR'S University of Mysore Jul 1996 – Jun 2000	Bachelor's in Mathematics

Career Continuity & Timeline

A transparent record of professional continuity, career transitions, and engineering growth across two decades.
(As of 15 Aug 2026)
Before DACS, I spent about a year after graduating in 2003 teaching discrete mathematics and C programming at an engineering college in Mysore; it is not counted in the industry totals below.

Company	Position	Tenure (From–To)	Reason for Leaving	Duration (y-m-d)	Career Break (y-m-d)
DACS	Software Engineer	2004/06/14 2005/05/23	Pursued long-term compiler engineering opportunities.	00y-11m-10d	00y-03m-22d
HP	Software Engineer	2005/09/19 2008/06/17	Accepted programming languages and compiler research opportunity.	02y-09m-00d	00y-00m-19d
SIEMENS	Lead Researcher	2008/07/07 2012/06/22	Accepted broader compiler engineering responsibilities.	03y-11m-16d	00y-00m-09d
AMD	Member of Technical Staff	2012/07/02 2013/03/28	Role concluded following corporate restructuring.	00y-08m-27d	00y-00m-17d
SAMSUNG	Chief Software Engineer	2013/04/15 2015/04/15	Planned career break for family responsibilities and long-term career reflection.	02y-00m-00d	01y-02m-18d
Phase 1 Completed — First decade of industry experience					
JIVOX	Software Architect	2016/07/04 2018/02/06	Pursued opportunities in large-scale machine learning systems.	01y-07m-03d	00y-03m-14d
QUBOLE	Member of Technical Staff	2018/05/21 2019/01/25	Returned to compiler engineering aligned with long-term career goals.	00y-08m-05d	00y-00m-09d
AMD	Senior Member of Technical Staff	2019/02/04 2022/06/17	Accepted an AI compiler engineering opportunity.	03y-04m-14d	00y-00m-02d
INTEL	AI Software Architect	2022/06/20 2026/08/15	Role concluded following corporate restructuring.	04y-01m-27d	
Phase 2 Completed — Twenty years of industry experience					
Total				20y-02m-12d	01y-11m-20d

Engineering Portfolio

A reverse-chronological record of the engineering problems, technical challenges, and corresponding solutions across a 20+ year compiler engineering career, from VLIW assembly-to-assembly code translation at DACS Software to Intel's AI/GPU compiler stack.

Intel, Bangalore		
AI Software Architect ● Jun 2022 – Aug 2026		
1	**PROBLEM**	Intel's AI accelerator represented tensor dimensions in a vector register ("dim5") holding up to five dimensions, with each dimension a 32-bit value matching the accelerator's 32-bit general-purpose registers (GPRs). As LLM weight counts grew, Intel widened each tensor dimension to 44 bits but kept the GPRs at 32 bits — and 44 bits has no natural, well-supported type in the LLVM/Clang type system. The compiler had to expose the wider dimensions to developers without (a) introducing a 44-bit language-level type, (b) disrupting the existing dim5 programming model, or (c) requiring developers to reason about hardware details at the language level.
	SOLUTION	• Designed the language-level model: each 44-bit tensor dimension exposed as a standard 64-bit type, since 64 bits map naturally onto a pair of 32-bit GPRs. The existing dim5 type thus transparently became a 5-dimension vector of 64-bit entries, letting the front-end and middle-end keep working with a single, standard-width type. • During instruction selection, lowered 64-bit tensor operations to the hardware's actual 44-bit width in two steps: (a) splitting each 64-bit value into a low 32-bit part and a high 12-bit part, mapped onto an even/odd pair of 32-bit GPRs, and (b) using the hardware's native low/high copy and data-movement instructions to move those parts between the GPR pair and the tensor register. • Implemented targeted post-instruction-selection optimization passes that removed redundant register moves introduced by this low/high emulation, keeping the generated code efficient despite the emulation overhead. • Building on the AI accelerator architecture understanding gained through this work, later wrote a tutorial paper on systolic arrays, the architecture behind many modern AI matrix engines.
2	**PROBLEM**	The 44-bit dim5 tensor-register support above shipped as production code, but, like any new feature, it surfaced bugs and rough edges as customer adoption grew.
	SOLUTION	• As the sole owner of the 44-bit dim5 support through customer adoption, stabilized it by resolving customer-reported issues, strengthening automated test coverage, and improving code maintainability through systematic refactoring. • Further optimized the 44-bit tensor code-generation passes to produce more efficient code.
3	**PROBLEM**	When the team was reassigned from AI accelerator compiler work to GPU compiler development, its engineers needed to ramp up quickly on GPU architecture and compiler requirements to become productive contributors.
	SOLUTION	• Led knowledge-sharing sessions that brought the team up to speed on core GPU architecture and key hardware-level details (e.g., warps, occupancy, memory hierarchy); later wrote this up as a tutorial paper. • Prototyped a divergence analysis pass for the GPU compiler and mentored the team through its concepts and implementation; later wrote a survey paper on the topic. • Researched IEEE 754 and emerging low-precision floating-point formats and briefed the team on their practical compiler implications.

4	**PROBLEM**	Automatically generating efficient GPU kernels from PyTorch computation graphs depends on robust graph fusion. The newly introduced MLIR Fuser had to be stabilized before it could serve as a reliable foundation for further fusion development.
	SOLUTION	• Ramped up quickly on MLIR and stabilized the MLIR Fuser by root-causing and fixing critical functional bugs across the compilation pipeline. • Improved compiler robustness through targeted refactoring and issue resolution across multiple phases of the MLIR-based graph fusion pipeline.
5	**PROBLEM**	Low-precision numerical formats (e.g., FP8, FP4) have become central to AI training and inference because they improve performance and memory efficiency. The MLIR Fuser therefore needed end-to-end support for them across the full MLIR compiler stack, including scale computation, quantization, and scaled matrix-multiply-accumulate (scaled MMA) lowering.
	SOLUTION	• As sole owner, designed and implemented end-to-end compiler support for low-precision floating-point types throughout the MLIR Fuser pipeline; later wrote a paper on modern low-precision formats for AI hardware. • Solved the hard problems this raised: sub-byte data representations, evolving GPU hardware capabilities, type representations that differ across AI frameworks, MLIR, LLVM, and GPU architectures, and fast-changing low-precision support across the AI stack. • Delivered scale-computation, quantization, and scaled-MMA lowering implementations, designed to extend cleanly as low-precision AI hardware continued to evolve.

AMD, Bangalore		
Senior Member of Technical Staff • Feb 2019 – Jun 2022		
1	**PROBLEM**	AMD was investing in a robust open-source GPU programming model to grow its ROCm ecosystem as an open alternative to CUDA. The Clang-based HIP-Clang compiler was central to that strategy and needed rapid stabilization as customer adoption grew.
	SOLUTION	• Stabilized the HIP-Clang compiler by resolving customer-reported issues and implementing requested language features. • Partnered with AMD's PyTorch team to establish HIP-Clang as the default compiler and toolchain for PyTorch's AMD GPU (HIP) backend. • Resolved HIP runtime blockers on the path to HIP-Clang's productization.
2	**PROBLEM**	AMD needed to support Cooperative Groups, a GPU programming model that lets kernels form and synchronize groups of threads both smaller and larger than the traditional thread block (work-group). Groups can exist at multiple levels of the execution hierarchy — warp-level sub-groups, thread blocks, and the whole grid — so the device library had to provide efficient, low-level abstractions for group handles and their member functions at each level.
	SOLUTION	• As sole owner, designed and implemented device-library support for Cooperative Groups across the full execution hierarchy — warp-level sub-groups, thread blocks, and the grid — in partnership with the GPU hardware team. • Implemented core handles and member functions — such as thread_rank(), num_threads(), and thread_index() — as efficient hardware abstractions that work consistently at every level of that hierarchy.

3	**PROBLEM**	On AMD GPUs, shared memory (LDS) is allocated per kernel, so a non-kernel (device) function cannot declare its own shared-memory variables; the compiler must ensure that memory exists when the function runs. AMD's existing module-lds-lowering pass did this by packing every shared-memory variable reachable from a kernel, across all of its call paths, into a single structure allocated once per kernel. This was functionally correct but sized for the worst case: large shared-memory arrays used only on rarely taken paths still consumed LDS space on every kernel launch, trading memory efficiency for functionality.
	SOLUTION	• As sole owner, designed and implemented an LLVM/AMDGPU compiler pass that replaced uses of LDS (shared-memory) globals in non-kernel functions with pointer indirection: each pointer is computed once in the owning kernel and reconstructed inside the non-kernel function. As a result, module-lds-lowering no longer had to pack the full LDS object into every kernel that could reach it, significantly reducing the per-kernel LDS memory footprint. • The patch was accepted and merged into upstream LLVM but later disabled by default after it caused regressions in OpenMP workloads.
4	**PROBLEM**	AMD's strategic collaboration with CERN required migrating CERN's GPU applications to the ROCm ecosystem with both functional correctness and competitive performance.
	SOLUTION	• As part of a small group of senior engineers, worked across multiple ROCm teams to bring CERN's GPU applications to functional correctness on the ROCm ecosystem. • Improved application performance through coordinated optimization work spanning both the AMD GPU compiler backend and CERN's application source code.
5	**PROBLEM**	As ROCm adoption grew in HPC, customers expected strong application performance, which required ongoing compiler improvements to resolve workload-specific bottlenecks.
	SOLUTION	• Root-caused and resolved customer-reported performance issues in HPC workloads through targeted compiler optimizations. • Improved generated-code quality to meet customer performance expectations by refining low-level optimizations, including register allocation and the instruction scheduler's clustering of independent memory operations.

Qubole, Bangalore		
Member of Technical Staff • May 2018 – Jan 2019		
1	**PROBLEM**	As deep learning frameworks rapidly gained adoption, Qubole saw an opportunity to extend its cloud-native data-lake platform beyond traditional big-data workloads by offering distributed deep learning as a managed service. This required evaluating how modern deep learning frameworks could integrate with Qubole's existing distributed infrastructure.
	SOLUTION	• Partnered with multiple engineering teams to evaluate how distributed deep learning frameworks could be integrated with Qubole's existing data-lake infrastructure. • Jointly built and validated a rapid prototype showing that the platform could support deep learning as a managed cloud service, informing architecture discussions on production feasibility.

Jivox, Bangalore		
Software Architect ● Jul 2016 – Feb 2018		
1	**PROBLEM**	As machine learning gained prominence in digital advertising, Jivox saw an opportunity to strengthen its ad-personalization platform with ML-based recommendations. These had to be integrated into the existing personalization engine without disrupting production stability.
	SOLUTION	• As sole owner, designed and integrated a collaborative-filtering-based recommender engine, implemented in Python, into the existing ad-personalization platform. • Iteratively refined the recommendation logic based on customer feedback and production performance data. • Stabilized the production system through systematic issue investigation and resolution.
2	**PROBLEM**	As user and product data volume grew rapidly, scaling the ad-personalization platform required sharding data efficiently across multiple database servers.
	SOLUTION	• As sole owner, designed and implemented a sharding algorithm that distributes user and product data across multiple database servers, and integrated it into the existing ad-personalization platform. • Improved the sharding infrastructure over time by resolving production issues and refining its logic.

Samsung, Bangalore		
Chief Software Engineer ● Apr 2013 – Apr 2015		
1	**PROBLEM**	Samsung's heterogeneous computing platform was evolving, and keeping its OpenCL compiler correct required broad improvements to the automated random-testing infrastructure across many features. Floating-point expression testing and pointer-expression testing were two of these areas; other engineers led work on the rest.
	SOLUTION	• As sole owner, designed and developed a floating-point expression testing methodology that validated numerical correctness within a configurable machine-epsilon tolerance. • As sole owner, designed and implemented a data-flow-analysis-based pointer-range analysis algorithm and integrated it into the random test generator (randomizer) so that it produced safe, semantically valid pointer-expression test cases; documented the work in an internal technical paper, and later revisited and published it as a technical paper.
2	**PROBLEM**	Improving the graphics rendering pipeline required identifying and resolving performance bottlenecks in OpenGL shaders.
	SOLUTION	• Collaborated with the graphics rendering team to identify and resolve OpenGL shader performance bottlenecks, improving overall rendering performance.

AMD, Bangalore		
Member of Technical Staff • Jul 2012 – Mar 2013		
1	**PROBLEM**	AMD was migrating its CPU compiler from the Open64 infrastructure to the emerging Clang/LLVM ecosystem, but several critical capabilities were still missing. OpenMP support was a key gap for production adoption, so designing and implementing it was an important milestone in the transition.
	SOLUTION	• As sole owner, proposed an OpenMP support implementation strategy for Clang/LLVM and presented it to the LLVM open-source community for technical review. • Engaged the LLVM community in evaluating design alternatives and converged on an approach aligned with LLVM's architectural direction. • Implemented initial Clang front-end support to recognize OpenMP constructs and generate the corresponding LLVM IR. The initiative was then discontinued due to changing business priorities, and responsibility for the upstream implementation passed to Intel.

Siemens, Bangalore		
Lead Researcher • Jul 2008 – Jun 2012		
1	**PROBLEM**	The Siemens Programming Languages Research Group helped Siemens business units solve challenges in programming languages, compiler technology, and software development methodology — translating research ideas into engineering solutions that product teams could adopt.
	SOLUTION	• Took ownership of building a new compiler front-end for the Structured Text (ST) language, which is used to program Siemens PLCs. • Joined the product business unit for the duration of the project, took on technical leadership of a team with no prior compiler background, and guided them to deliver the product within the required business timeline. • Owned the full engineering lifecycle: requirements specification, architecture and design, prototyping, team mentoring and collaboration, implementation, and delivery.
2	**PROBLEM**	As heterogeneous computing emerged as a promising paradigm, Siemens launched exploratory research into whether application workloads could be automatically analyzed and transformed to run efficiently on heterogeneous platforms such as GPUs.
	SOLUTION	• Collaborated with Siemens research teams across multiple global locations to investigate heterogeneous programming models and source-to-source transformation techniques. • Evaluated multiple architectural approaches through rapid prototyping, including an in-depth study of the CUDA ecosystem and GPU programming methodologies. • Assessed the technical feasibility and trade-offs of the proposed architecture before the research initiative was discontinued due to changing business priorities.
3	**PROBLEM**	As Intel's Many Integrated Core (MIC) architecture emerged as a potential high-performance computing platform, Siemens wanted to evaluate how well it suited future business and engineering workloads.
	SOLUTION	• Partnered with Intel's programming research team to evaluate the capabilities of the Intel MIC architecture. • Ported representative Siemens workloads to the MIC platform and analyzed their performance characteristics. • Documented the evaluation in a technical paper co-authored with Intel's research team.

HP, Bangalore		
Software Engineer • Sep 2005 – Jun 2008		
1	**PROBLEM**	As the C++ language evolved rapidly, HP's in-house C++ compiler front-end struggled to keep pace due to architectural limitations, making it increasingly difficult to support new C++ features while preserving compiler stability.
	SOLUTION	• Maintained and extended HP's in-house PA-RISC C++ compiler front-end. • Resolved parser issues in the traditional Lex/Yacc-based front-end while extending it to accept modern C++ syntax. • Investigated and resolved semantic issues in C++ templates and other advanced language features.
2	**PROBLEM**	Because maintaining the legacy C++ front-end had become increasingly difficult, HP began migrating to the industry-standard EDG C++ front-end. The key engineering challenge was preserving HP's PA-RISC ABI compatibility while integrating the target-independent EDG front-end.
	SOLUTION	• Collaborated with senior compiler engineers on the EDG integration effort. • Contributed to the design and successful implementation of an ABI-compatibility bridge that lowered EDG-generated ASTs into HP's native ASTs while preserving ABI correctness. • Contributed to AST transformation, code refactoring, bug fixing, and compiler cleanup required for the migration. • Helped successfully integrate the EDG front-end into HP's C++ compiler toolchain; the work is documented in a technical paper.

DACS Software, Bangalore		
Software Engineer • Jun 2004 – May 2005		
1	**PROBLEM**	As part of DACS Software's collaboration with CEVA, an existing assembly-to-assembly translation tool had to be extended to migrate applications from the TEAK DSP architecture to the CEVA DSP architecture. Because CEVA uses a VLIW architecture while TEAK uses a traditional dual-MAC architecture, a straightforward one-to-one instruction mapping could not produce efficient target code.
	SOLUTION	• Was part of the engineering team extending the assembly-to-assembly translation tool for TEAK-to-CEVA DSP migration. • Designed and implemented the instruction-packing strategy that grouped translated instructions into VLIW packets, establishing the baseline for VLIW code generation in the migrated applications. • Investigated and implemented successive optimization techniques to improve the quality and performance of the generated code. • Under tight project timelines, also hand-ported selected performance-critical TEAK applications to the CEVA DSP architecture.